Computer Vision

Assignment 1

Tamer Abdullatif 211798715

Odai Agbaria 212609440

Part A – Question 1: Homography Computation

Let $x = (u, v, 1)^T$ and $x' = (u', v', 1)^T$ be corresponding points in the source and destination images. A projective transformation is represented by a 3×3 homography H:

$$x' \cong Hx, \qquad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} = \begin{bmatrix} H_1 \\ H_2 \\ H_3 \end{bmatrix}.$$

Here $\cong$ means equality up to scale. Dividing by the third homogeneous coordinate gives

$$u' = \frac{H_1 x}{H_3 x}, \qquad v' = \frac{H_2 x}{H_3 x}.$$

Linear system. Multiplying by the denominators and rearranging:

$$u' H_3 x - H_1 x = 0, \qquad v' H_3 x - H_2 x = 0.$$

Define $h = (h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33})^T$. Each matching pair contributes two rows:

$$\underbrace{\begin{bmatrix} -u & -v & -1 & 0 & 0 & 0 & u'u & u'v & u' \\ 0 & 0 & 0 & -u & -v & -1 & v'u & v'v & v' \end{bmatrix}}_{A_i} h = 0.$$

For n pairs, stacking the A_i gives $Ah = 0$ with $A \in \mathbb{R}^{2n \times 9}$. This is an $Ax = b$ system with $b = 0$. Since H is defined up to scale, it has eight degrees of freedom, so **at least four nondegenerate pairs** are required (for four pairs, no three points should be collinear in either image).

Alternatively, fixing $h_{33} = 1$ (when $h_{33} \neq 0$) gives the nonhomogeneous form:

$$\begin{bmatrix} u & v & 1 & 0 & 0 & 0 & -u'u & -u'v \\ 0 & 0 & 0 & u & v & 1 & -v'u & -v'v \end{bmatrix} \underbrace{[h_{11} \quad h_{12} \quad h_{13} \quad h_{21} \quad h_{22} \quad h_{23} \quad h_{31} \quad h_{32}]^T}_{h_8} = \begin{bmatrix} u' \\ v' \end{bmatrix}.$$

Recovering H. For the homogeneous system we avoid the trivial solution by imposing $\| h \| = 1$ and solve

$$\min_{\|h\|=1} \| Ah \|^2 = \min_{\|h\|=1} h^T A^T A h.$$

If $A^T A e_i = \lambda_i e_i$, where $\lambda_1 \leq \cdots \leq \lambda_9$, and $h = \sum_i \mu_i e_i$ with $\sum_i \mu_i^2 = 1$, then

$$\| Ah \|^2 = \sum_i \mu_i^2 \lambda_i \geq \lambda_1.$$

Thus h is an eigenvector of $A^T A$ with the smallest eigenvalue. In practice, use **SVD**, $A = U \Sigma V^T$: take the right singular vector for the smallest singular value and reshape it row-by-row into $H \in \mathbb{R}^{3 \times 3}$. If $h_{33} \neq 0$, we may normalize $H \leftarrow H / h_{33}$.

Part A – Question 2: Estimating the Homography

The function compute_homography_naive() builds the linear system derived in Question 1. Each source–destination point pair adds two rows to A:

$$\begin{bmatrix} -x_i^{\mathrm{T}} & 0^{\mathrm{T}} & u'_i x_i^{\mathrm{T}} \\ 0^{\mathrm{T}} & -x_i^{\mathrm{T}} & v'_i x_i^{\mathrm{T}} \end{bmatrix}$$

For N matches, A has size 2N × 9. The vector h contains the three rows of H. Since H is defined up to scale, we solve Ah = 0 with ‖h‖ = 1 to avoid the zero solution.

$$min\ ||Ah||^2\ \ subject\ to\ \ ||h||\ =\ 1$$

Using SVD, A = UΣV^T, we take the right singular vector corresponding to the smallest singular value (the last row of V^T) and reshape it into the 3 × 3 matrix H. At least four nondegenerate matching pairs are needed; additional pairs give an algebraic least-squares estimate.

Part A – Question 3: Homography from Perfect Matches

Using the 20 matching pairs in matches_perfect.mat, the computed homography (normalized to ‖H‖ = 1) is:

$$H\ =\ \begin{bmatrix} 1.12313781e-3 & 1.64757662e-4 & -9.99919585e-1 \\ 1.05117244e-5 & 1.05462483e-3 & -1.25622165e-2 \\ 2.96940746e-7 & 4.35706350e-8 & 7.82907867e-4 \end{bmatrix}$$

Because homographies are defined up to scale, dividing all entries by h_{33} gives the equivalent matrix:

$$H\ /\ h_{33}\ =\ \begin{bmatrix} 1.4346 & 0.2104 & -1277.19 \\ 0.0134 & 1.3471 & -16.046 \\ 3.793\ \times\ 10^{-4} & 5.565\ \times\ 10^{-5} & 1 \end{bmatrix}$$

Both matrices describe the same mapping from src to dst. The first is the direct SVD result; the second is scaled so that h_{33} = 1.

Part A2 – Forward Mapping (Questions 4–7)

4. Forward Mapping — Slow Implementation

The function **compute_forward_homography_slow()** visits each source pixel (column c, row r), transforms $(c, r, 1)^T$ using the Homography from Question 3, divides by the third coordinate, and rounds to the nearest integer. It copies the source color only when the mapped position is inside the destination image.

The output is initially black and has the destination dimensions (1088 × 1452). Only the part of the warped source falling within this frame is visible; much of the source maps beyond its left boundary.

5. Forward Mapping — Fast Implementation

The function **compute_forward_homography_fast()** creates a meshgrid of all source coordinates and stacks them into a 3 × (H·W) matrix of homogeneous points. One matrix multiplication transforms all points at once; normalization, rounding and NumPy indexing then place the valid pixels in the output without Python loops.

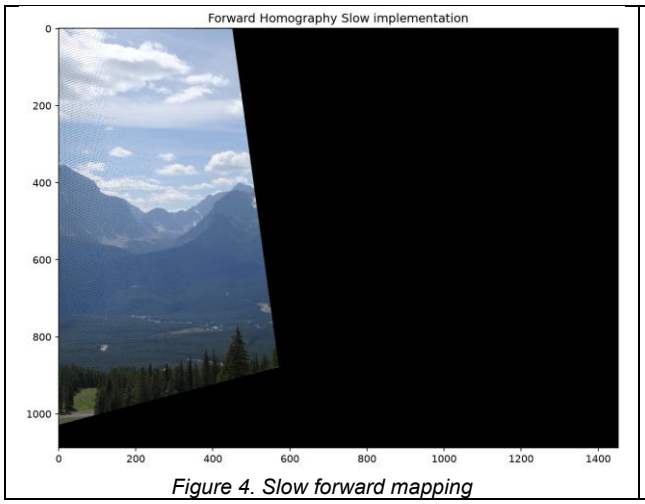


Figure 4. Slow forward mapping

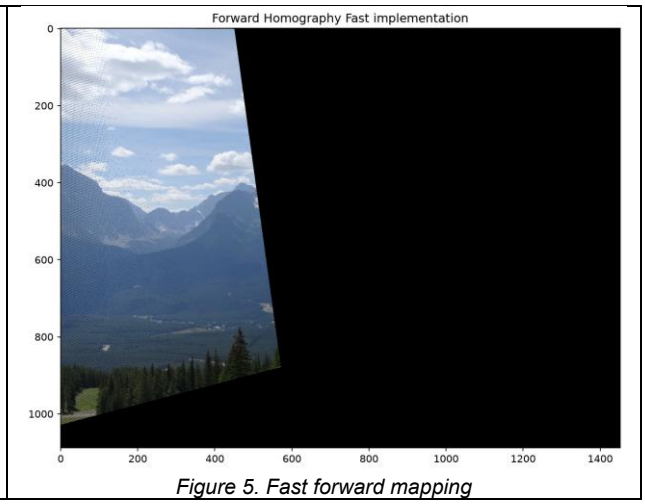


Figure 5. Fast forward mapping

Both outputs show the same warped region and the characteristic small black gaps. The vectorized version took 0.06 s compared with 5.75 s for the loop version about X96 faster on our machine. In our run, the two images were identical pixel for pixel.

6. Problems with Forward Mapping

Forward Mapping applies H to each source pixel and writes its color at the corresponding destination location. The resulting image illustrates two limitations discussed in class.

Fractional coordinates and overlaps. Projected positions are usually not integers, so we round to the nearest destination pixel. Several source pixels can then write to one output position. Of 566,771 source pixels mapped inside the frame, only 484,727 distinct destination pixels were written: 82,044 overlapping assignments (14.5%).

Holes. When the transformation enlarges part of the image, neighboring source pixels can land more than one destination pixel apart. Some output pixels are never assigned a color and remain black. Our measurement found approximately 7,590 holes (about 1.5%) in the warped region. The rate was higher in enlarged areas (5.9%) than in reduced areas (0.5%).

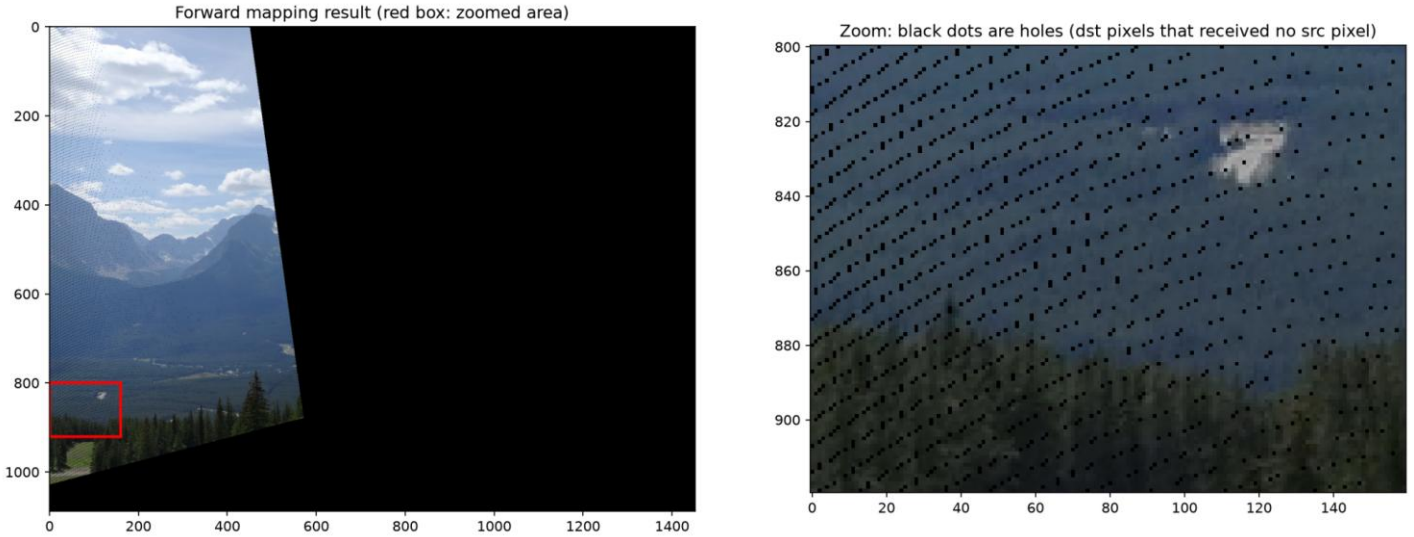


Figure 6. Forward-mapped image (left) and enlarged view of unfilled pixels (right).

The small black dots are mapping holes; the large black background to the right lies outside the warped image's coverage and is not counted as a hole. Backward Mapping (Part C) instead visits destination pixels and interpolates their source values, avoiding these internal gaps in valid regions.

7. Forward Mapping with Imperfect Matches

We repeated the fast forward warp using matches.mat, which contains 20 correct correspondences and 5 outliers. The naive SVD estimate using all 25 pairs gave the following unit-norm Homography:

$$H = \begin{bmatrix} -5.69473545e-4 & -1.27553946e-4 & 6.07820712e-1 \\ -6.08182336e-4 & -4.71576145e-4 & 7.94073110e-1 \\ -9.21258229e-7 & -3.74324571e-7 & 9.71767385e-4 \end{bmatrix}$$

The naive fit is strongly affected by the five incorrect matches (roughly 117–281 pixels from their correct mapped locations). With max_err = 25 px, only 4/25 pairs (16%) qualify as inliers, compared with 20/25 (80%) for the correct model.

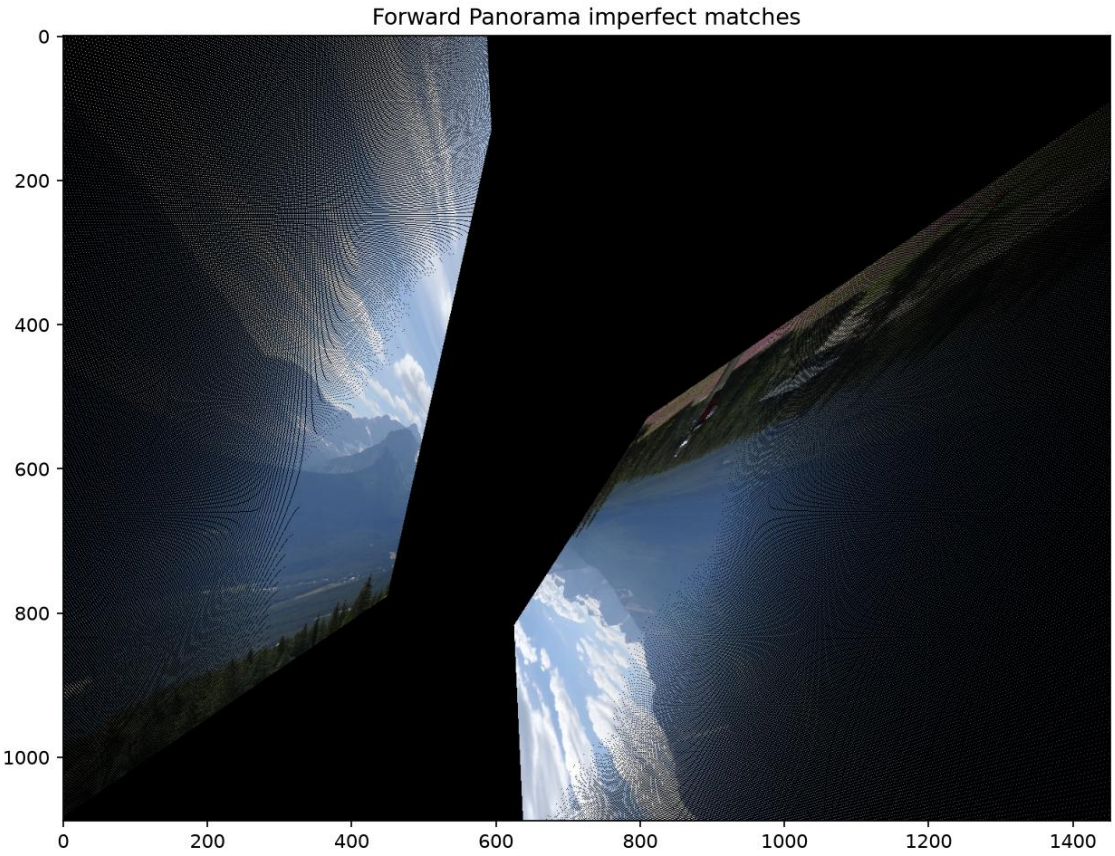


Figure 7a. Destination-sized forward warp using the naive Homography from imperfect matches.

Unlike Question 5, the image is severely distorted and split into two regions, one appearing inverted. The denominator $w = h_{31}u + h_{32}v + h_{33}$ becomes zero on a line crossing the source image. Points near this line are sent very far away; on opposite sides, the mapped image can appear separated or reversed.

7. Forward Mapping with Imperfect Matches (continued)

Large-range visualization. The full transformed image would need an impractically large canvas: mapped coordinates span about 151 million pixels horizontally and 344 million vertically. The figure below uses the middle 98% of projected coordinates (1st–99th percentiles), a window of approximately 9,575 × 10,100 pixels, reduced by 6.7×. The red rectangle marks the destination frame.

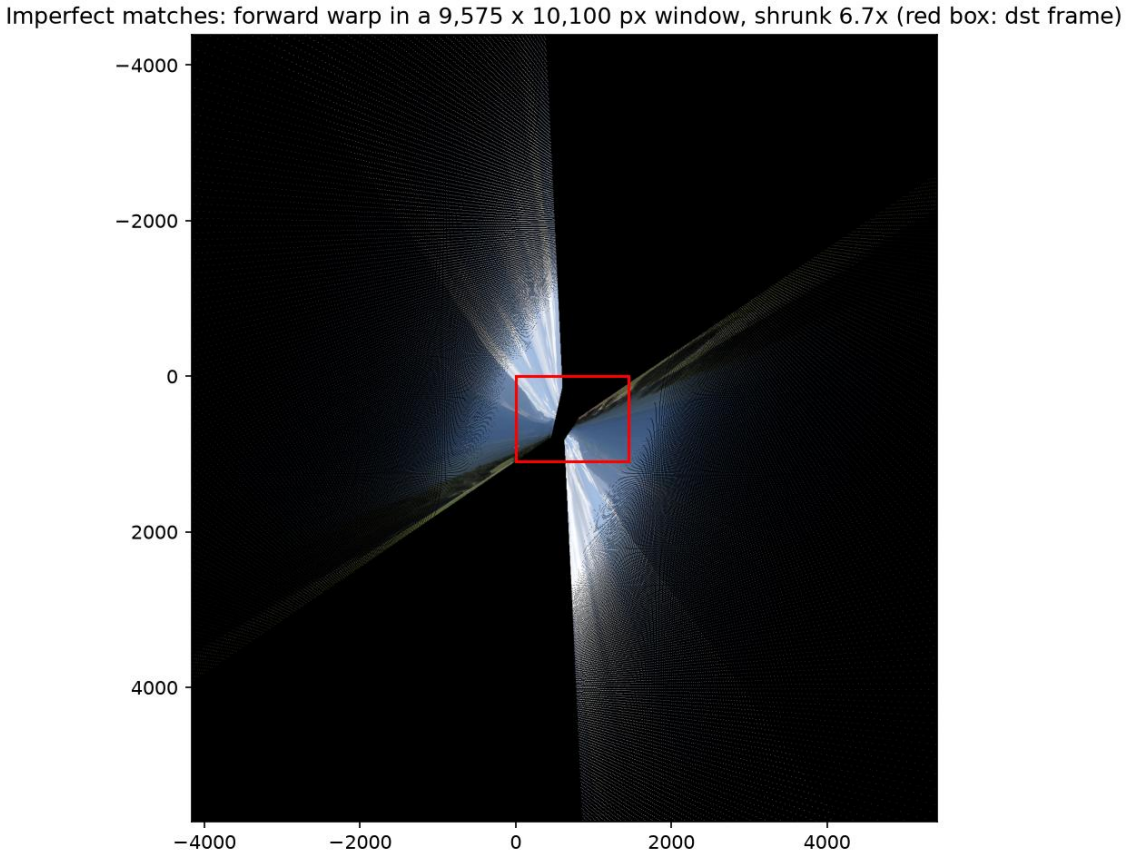


Figure 7b. Reduced view of the large warped coordinate window; red box = destination frame.

Comparison with Question 6. The incorrect Homography amplifies the same Forward Mapping artifacts: approximately 60% of pixels in the warped region are holes (up to about 89% where magnification exceeds 5×), and roughly 60% of in-frame source assignments are overwritten. The corresponding figures with perfect matches were 1.5% and 14.5%.

These results show why using every match equally can fail: even a few outliers can greatly distort the Homography. We address this in Part B using RANSAC.

Part B – Dealing with Outliers (Questions 8–12)

8. Evaluating a Homography – test_homography()

The function checks how well a homography maps the source matching points to the destination points. We first convert each source point to homogeneous coordinates, apply H, and divide by the third coordinate to get its projected 2D location.

$$\tilde{x}_i = (u_i, v_i, 1)^T, \quad \hat{x}'_i = (H\,\tilde{x}_i)_{1:2} / (H\,\tilde{x}_i)_3$$

The Euclidean distance d_i between the projected point and its destination match determines whether the pair is an inlier (d_i < max_err). Let D be the number of inliers among N matches. We report the inlier fraction and the mean squared distance over those D points:

$$\text{fit_percent} = D / N, \qquad \text{dist_mse} = (1 / D) \sum d_i^2 \;\; (i \in I)$$

If no points meet the threshold, the function returns dist_mse = 10^9, as required in the function documentation.

9. Finding Inliers – meet_the_model_points()

This function reuses the point-to-point distances from Question 8. It creates a Boolean mask for distances below max_err and applies that mask to both sets of matching points. The returned source and destination arrays contain only the corresponding inlier pairs, each with shape 2 × D, where D is the number of inliers.

10. Number of RANSAC Iterations

We are given 30 matches, with an expected inlier ratio w = 0.8. Each RANSAC trial randomly selects n = 4 matches, so the probability that all four are inliers is approximated by $w^4 = 0.4096$. For confidence p, the required number of trials is:

$$k = \lceil \log(1 - p) / \log(1 - w^4) \rceil$$

For p = 0.90, k = 5 trials; for p = 0.99, k = 9 trials. Exhaustive search would require C(30, 4) = 27,405 different groups of four matches. Random sampling therefore needs far fewer trials to reach the requested confidence.

11. Robust Homography – compute_homography()

We use RANSAC to estimate a homography even when some matches are incorrect. The implementation sets p = 0.99, d = 0.5, n = 4, w = inliers_percent and t = max_err. It runs $k = \lceil \log(1 - p) / \log(1 - w^4) \rceil + 1$ iterations (10 when w = 0.8).

1. Randomly select four matching pairs and estimate a homography with compute_homography_naive().
2. Find the inliers using meet_the_model_points() and the distance threshold t.
3. If more than 50% of all pairs are inliers, recompute H using the full inlier set.
4. Evaluate the refined model with test_homography(), keeping the one with the lowest inlier MSE.

The function returns the best saved homography. If no trial passes the 50% threshold, it falls back to fitting a model to the largest consensus set found.

12. Homography Estimation Using RANSAC

Using all 25 matches in matches.mat with inliers_percent = 0.8 and max_err = 25 px, compute_homography() returned the unit-norm matrix below. Its sign is opposite to the Question 3 result, but H and −H define exactly the same projective transformation.

$$H = \begin{bmatrix} -1.12313781e-3 & -1.64757662e-4 & 9.99919585e-1 \\ -1.05117245e-5 & -1.05462483e-3 & 1.25622165e-2 \\ -2.96940746e-7 & -4.35706350e-8 & -7.82907867e-4 \end{bmatrix}$$

Dividing by h_{33} gives the same normalized mapping as in Question 3 (h_{33} = 1). RANSAC accepted the 20 consistent pairs and rejected the 5 outliers; the quantitative difference from the naive estimate is substantial:

Method	Inlier fraction	Inlier MSE (px²)
Naive (Question 7)	0.16 (4/25)	456.3
RANSAC (Question 12)	0.80 (20/25)	4.64

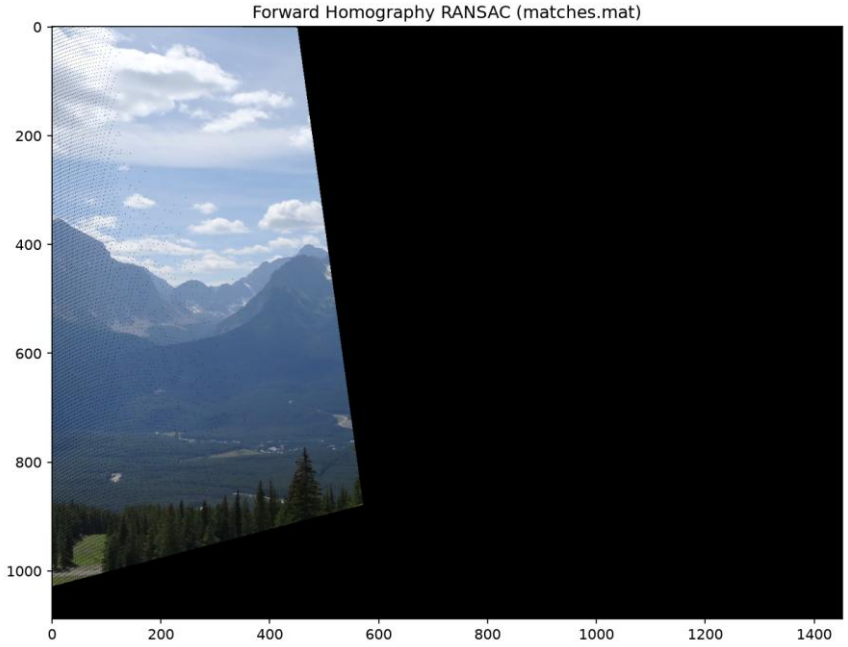


Figure 12a. Forward mapping using the robust RANSAC Homography.

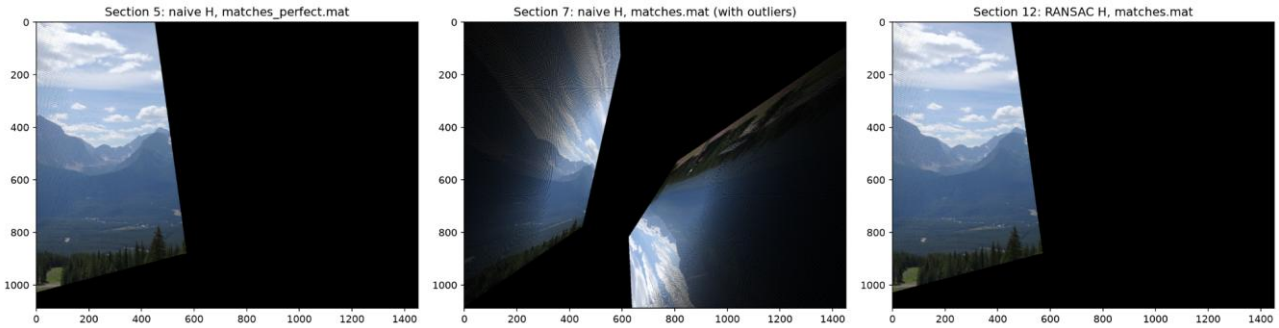


Figure 12b. Same destination frame: perfect matches (Q5), naive fit with outliers (Q7), and RANSAC (Q12).

The RANSAC warp is visually the same as the correct warp in Question 5, while the naive solution from Question 7 is heavily distorted. RANSAC removes the effect of the mismatches in this run, although the black holes and overlapping pixel assignments remain because they are inherent to Forward Mapping.

Because RANSAC samples randomly, results can vary. In a 10,000-run experiment, the 20-correct-inlier set was recovered in approximately 96% of runs; this observed final-model rate is different from the theoretical 99% sampling confidence.

Part C – Panorama Creation (Questions 13–16)

13. Backward Mapping and Bilinear Interpolation

Forward mapping (Q12) sends each source pixel to a destination position. Because the projected coordinates are rounded to integer pixels, several source pixels may land on the same destination pixel while other destination pixels are never visited. This produces small black holes inside the transformed image. In backward mapping, we instead visit every destination pixel and use the inverse homography to determine where to sample the source image.

$$p_src \sim H^{-1} p_dst$$

The source coordinates are usually fractional. We therefore use bilinear interpolation: the output color is a weighted combination of the four nearest source pixels, with weights determined by the fractional horizontal and vertical offsets. This avoids gaps caused by rounding and gives smoother local intensity transitions.

$$I(x,y) = (1-a)(1-b)I_{00} + a(1-b)I_{10} + (1-a)bI_{01} + abI_{11}$$



Figure 13a. Forward mapping (Q12, left) and backward mapping with bilinear interpolation (Q13, right).

At normal display size, the small forward-mapping holes can be difficult to notice. Figure 13b enlarges a corresponding region of both outputs so that the scattered dark pixels in the forward warp can be compared with the continuous backward-warped result.



Figure 13b. Enlarged corresponding image regions; forward-mapping holes are more visible on the left.

The measured runtime was 0.153s for forward mapping and 0.357s for backward mapping (approximately 2.33 times as long). Backward mapping performs additional work: for each destination pixel it transforms coordinates, locates four source neighbors, and computes interpolation weights and a weighted color. This extra computation produces a more complete and visually smoother warp; it does not, itself, guarantee that the estimated homography is geometrically more accurate. Large black areas outside the projected source-image coverage are expected in both methods.

14. Adjusting the Backward Homography for Padding

The panorama canvas must accommodate both images. The function find_panorama_shape() computes its dimensions and the padding needed to place the destination image within it. Because the backward homography expects coordinates relative to the destination image, panorama coordinates must first be shifted back by the left and top padding.

$$T = [[1, 0, -pad_left], [0, 1, -pad_up], [0, 0, 1]]$$

$$H_adjusted = H_backward \cdot T$$

The order of multiplication is essential: T first converts panorama coordinates into destination coordinates; H_backward then maps them to the source image. This ensures that the warped source and the destination image share the same coordinate system.

15. Constructing the Panorama

The panorama() function estimates the source-to-destination homography using RANSAC, obtains the panorama canvas dimensions from the projected source corners, and computes the inverse homography. After adjusting it for padding (Q14), backward mapping (Q13) fills the panorama canvas with the transformed source image. The destination image is then placed at its padded coordinates.

In overlapping areas, the destination image takes priority, as required. We use a spatial mask for its rectangular footprint rather than testing whether pixel values are zero; a genuinely black pixel in the destination image must still be preserved.

16. Result Using the Provided Images

We ran main.py on the provided images with an expected inlier fraction of 0.8 and a maximum reprojection error of 25 pixels. RANSAC retained 20 of the 25 correspondences (80% inliers) and achieved an inlier MSE of 4.6420. The panorama canvas was 2726 × 1306 pixels with 1274 pixels of left padding and 14 pixels of top padding. The measured panorama computation time for this run was 1.620 s.

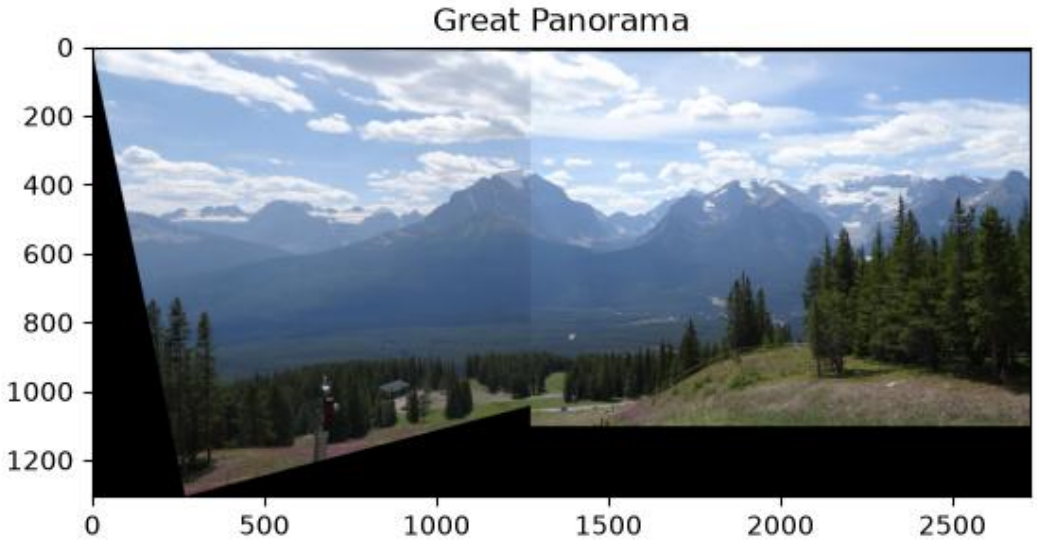


Figure 16. Panorama constructed from the provided images using RANSAC and backward mapping.

The mountain landscape is aligned across the overlap. A vertical seam remains visible because the implementation favors destination pixels without blending; minor appearance differences between the photographs are therefore not smoothed away. The black triangular borders mark areas of the rectangular panorama canvas that neither image covers. They are not the internal sampling holes observed in forward mapping.

17. Panorama Using Our Own Images

We used two overlapping photographs from the OpenCV stitching dataset (boat1.jpg and boat2.jpg). Both images were resized to 1280 × 853 pixels and saved as src_test.jpg and dst_test.jpg. We manually selected 25 correspondence pairs, including three intentionally incorrect pairs (12% outliers).



Figure 17a. The 25 manually selected point pairs. Green marks inliers and red marks outliers.

With a 25-pixel reprojection-error threshold, an independent RANSAC check identified 22 inliers out of 25 pairs (88%) and three outliers (12%). The two image views share stable features on the church tower, bridge, and waterfront buildings.

The four figures below are the actual outputs obtained by running main.py with the boat images. Forward mapping leaves small unfilled pixels, whereas backward mapping uses interpolation to fill the valid warped region more smoothly.

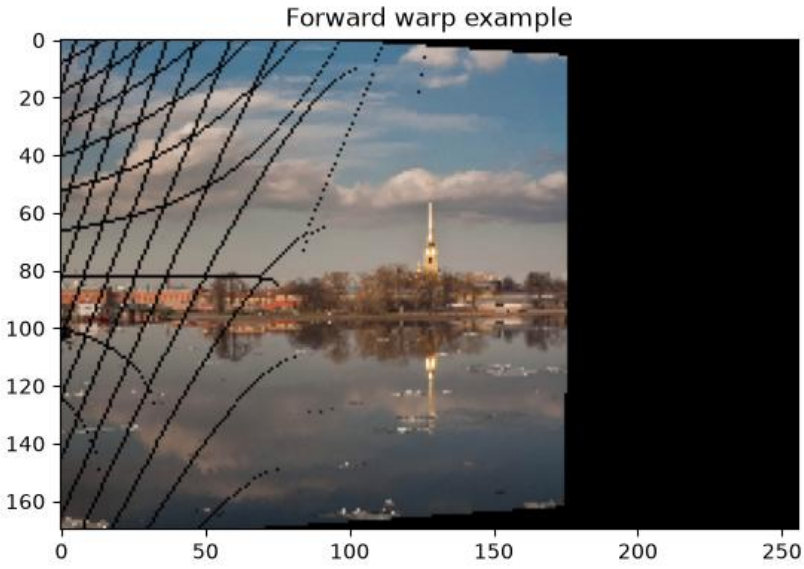


Figure 17b. Forward warp example on the boat images.

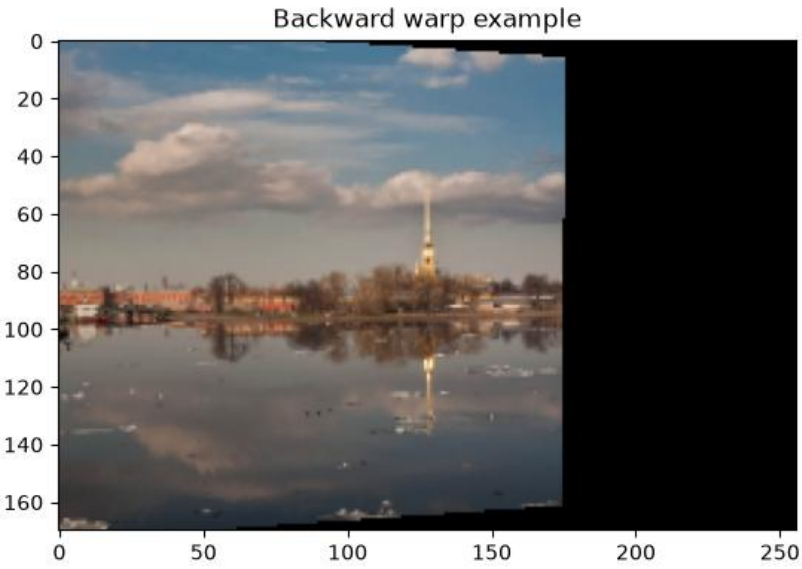


Figure 17c. Backward warp example on the boat images.

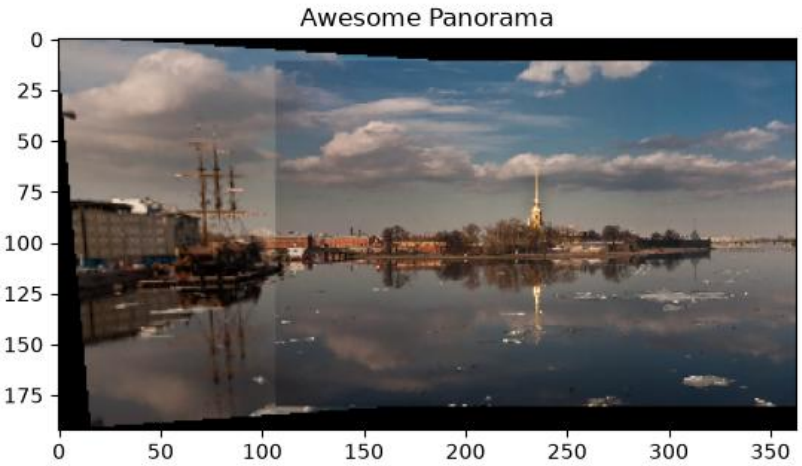


Figure 17d. Awesome Panorama: Boat 1 and Boat 2 stitched in the normal direction.

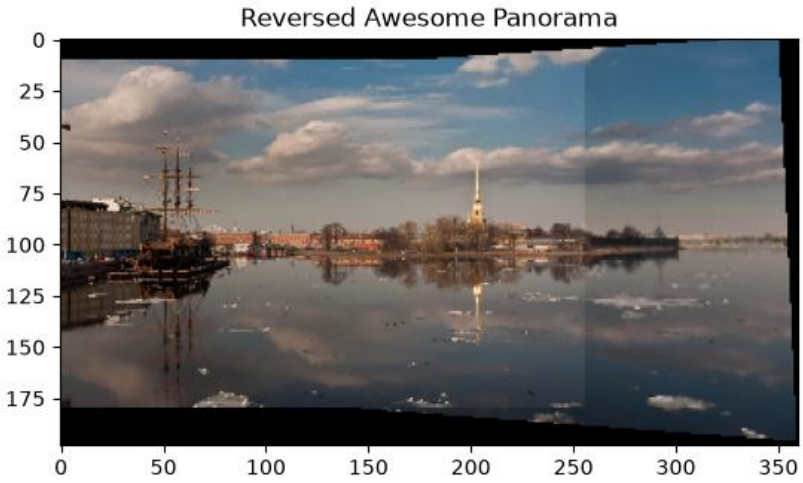


Figure 17e. Reversed Awesome Panorama: stitching in the opposite direction.

The two stitched panoramas align the church tower and waterfront structures across the overlap. A vertical seam is visible where the images meet because the implementation uses destination-image priority without blending. Black borders correspond to areas outside the coverage of both images.

Runtime: the terminal reported Student Panorama 0.0537s and 0.0550s for the two boat-image panorama calls (normal and reversed, respectively). The earlier Panorama 2.1590 s belongs to the provided-image test, not to these boat-image runs. These times reflect the execution paths measured by main.py; they should not be interpreted as timings for all of the manual point selection or the entire script.